# コンパイラ (10)

2026/11/6

# コード最適化

# コード最適化

実行の意味を変えることなく実行を高速化すること。

実行の意味：入力と出力
最適化の対象によって入力と出力は変わる。
プログラム解析が必要

- 機械独立なコード最適化
  =3 番地コードのレベルで実行を効率化
- 機械依存なコード最適化
  =対象マシンのアーキテクチャに依存
  (VLIW、マルチコアなど)

# 基本ブロック

## 最後以外分岐のない文の並び

```
prod := 0;
i := 0;
while (i < 20) begin
  prod := prod + a[i]*b[i];
  i := i + 1
end;
```

```
 prod = 0
 i = 0

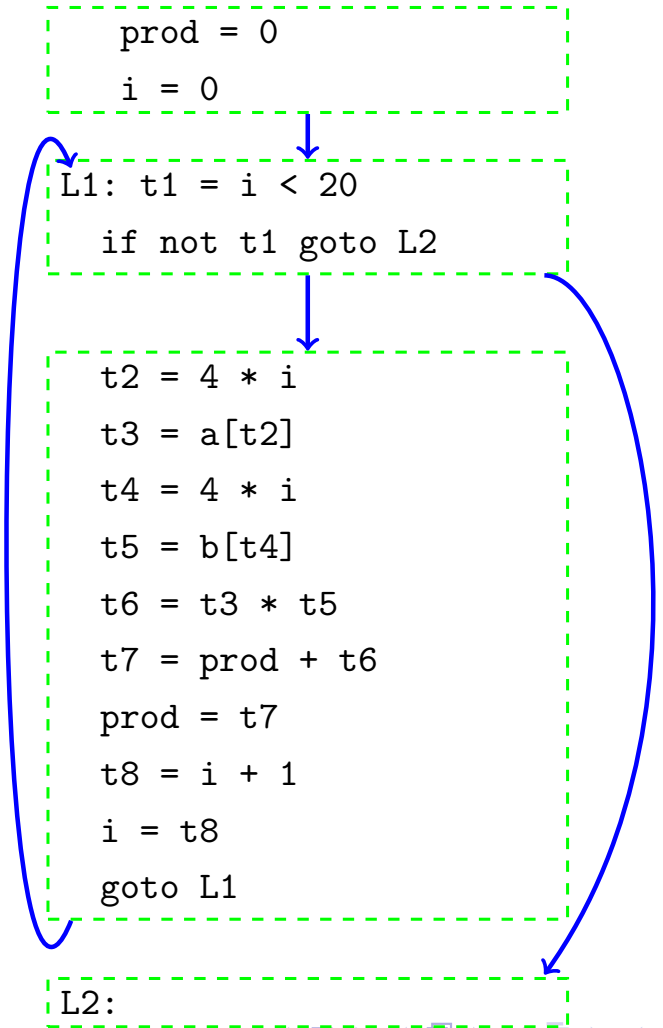
L1: t1 = i < 20
  if not t1 goto L2


  t2 = 4 * i
  t3 = a[t2]
  t4 = 4 * i
  t5 = b[t4]
  t6 = t3 * t5
  t7 = prod + t6
  prod = t7
  t8 = i + 1
  i = t8
  goto L1
```

# 基本ブロック

## 最後以外分岐のない文の並び

```
prod := 0;
i := 0;
while (i < 20) begin
  prod := prod + a[i]*b[i];
  i := i + 1
end;
```



# 局所最適化

基本ブロック内だけを見て最適化すること

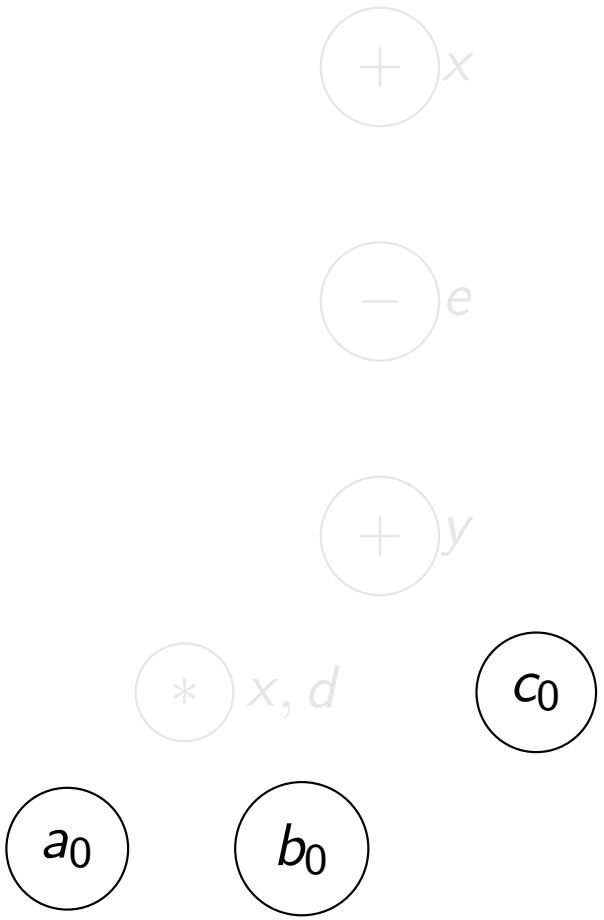
覗き穴最適化 (Peep-hole optimization)

- 定数伝播 (constant propagation)
  定数の計算、参照を定数そのものでおきかえる
- 共通部分式の削除 (common subexpression elimination)
  共通の部分式を抽出して結果を一時変数に保存するようにして、二回目以降計算しない。
- コピー伝播 (copy propagation)
  余分な変数の仲介を減らす。
  データ依存性を減らして命令の並列発行が可能になる場合がある。
- 不要コードの除去 (deadcode elimination)
  使われない変数に関する計算を消去する。
- 代数的性質の利用
  演算子が持つ代数的性質 (可換則、結合則、分配則、計算のためのコストなど)

# DAG による局所最適化

DAG=Directed Asyclic Graph
閉路 (ループ) をもたない有効グラフ
基本ブロック内のデータ依存性を一意的に表現
DAG の構成アルゴリズム 6.6

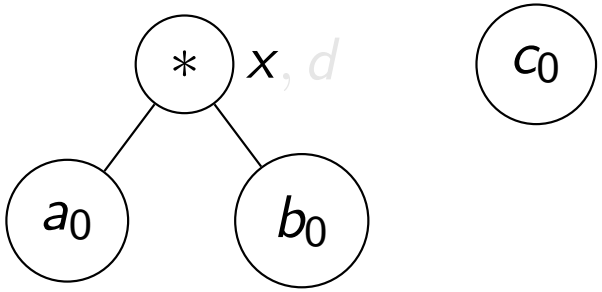
# DAG の構成



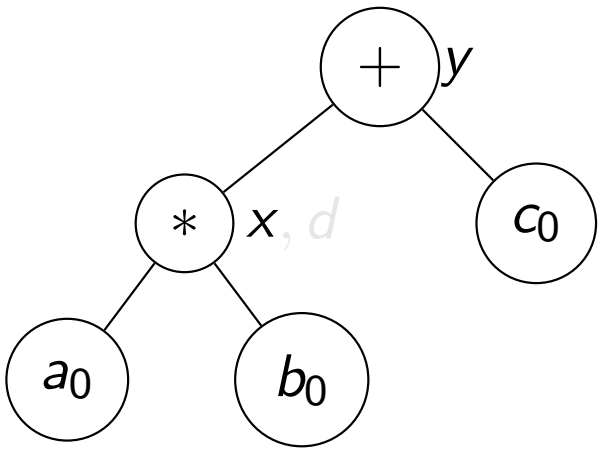

```
x=a*b
y=x+c
d=a*b
e=d-c
x=d+e
```

# DAG の構成

x=a*b

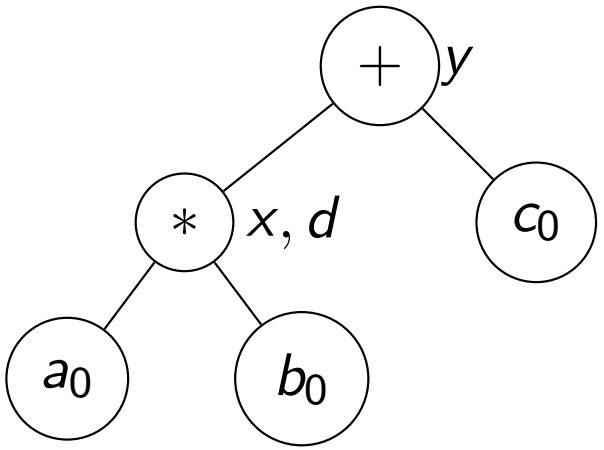


# DAG の構成



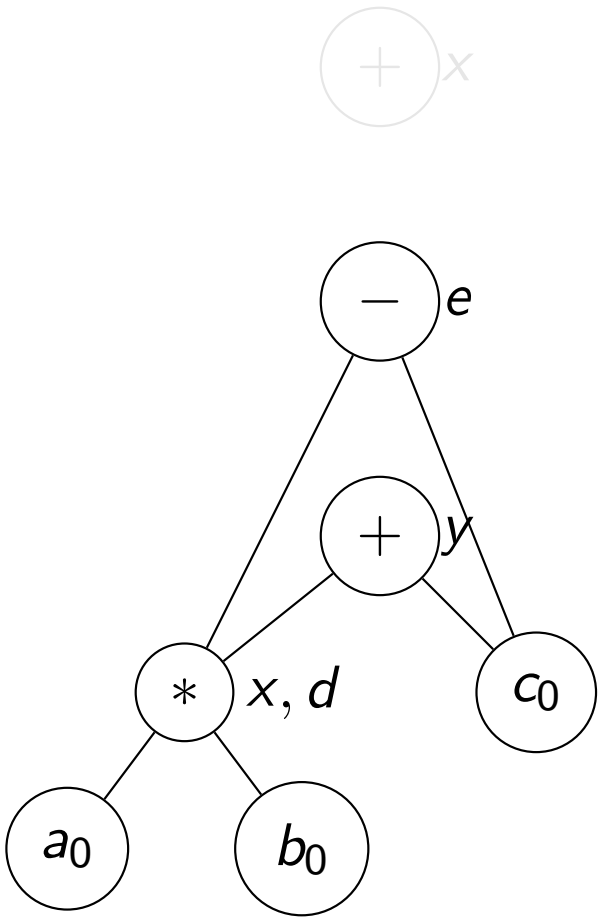

```
x=a*b
y=x+c
```

# DAG の構成



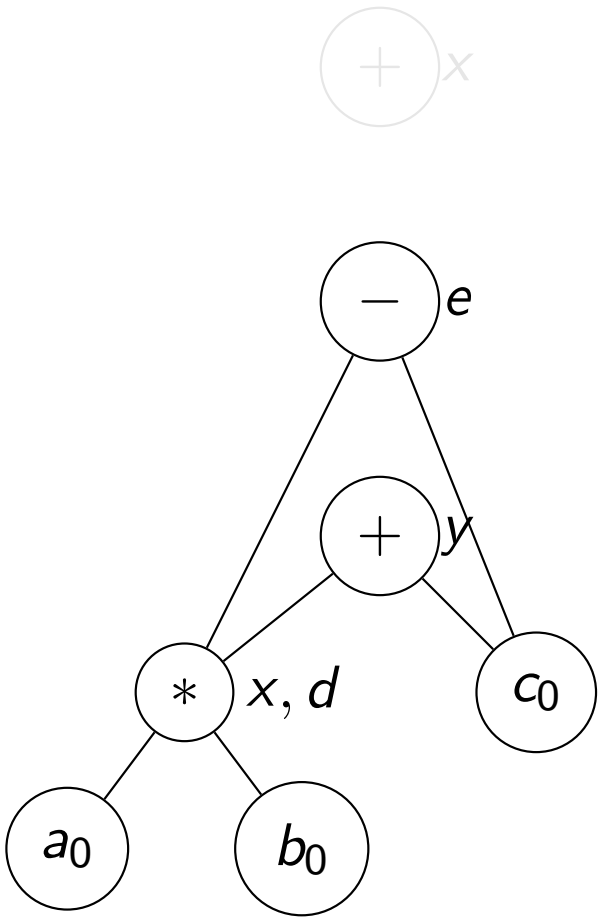

```
x=a*b
y=x+c
d=a*b
```

# DAG の構成



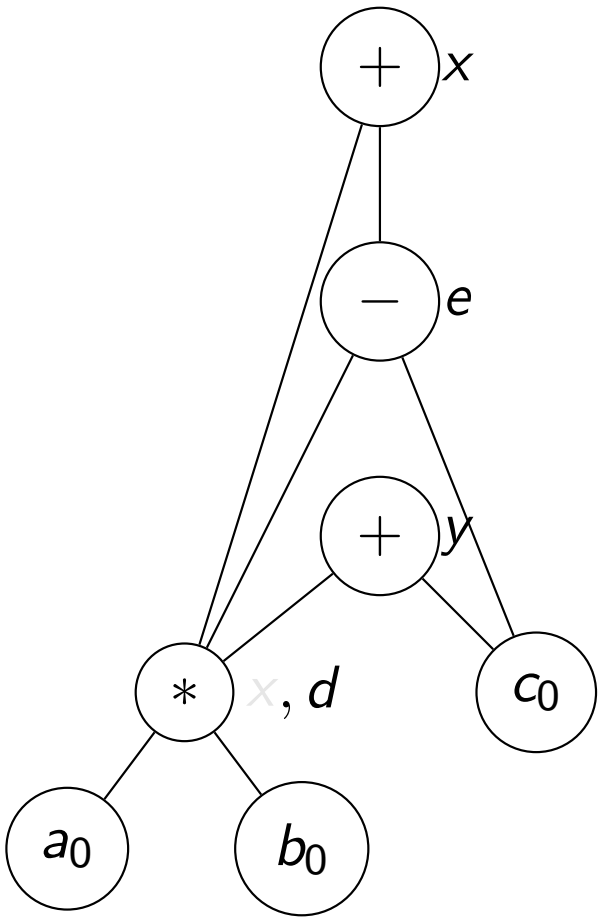

```
x=a*b
y=x+c
d=a*b
e=d-c
```

# DAG の構成




```
x=a*b          x=a*b
y=x+c    ⇒     y=x+c
d=a*b          d=x
e=d-c          e=d-c
```

# DAG の構成

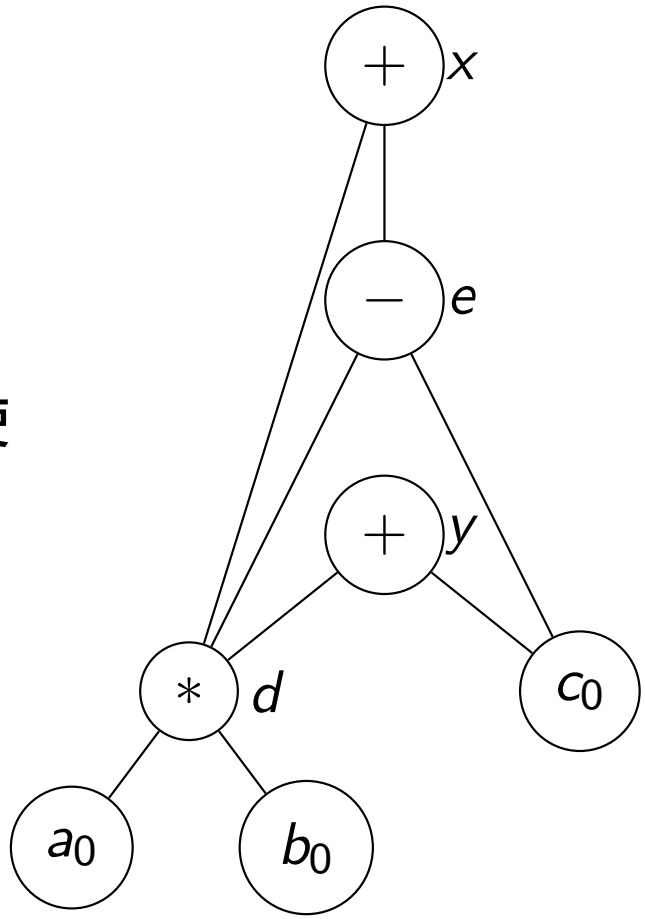



```
x=a*b
y=x+c
d=a*b
e=d-c
x=d+e
```

```
d=a*b
y=d+c
e=d-c
x=d+e
```

# 不要コードの消去

これ以降、$x$ しか使用しないとすると

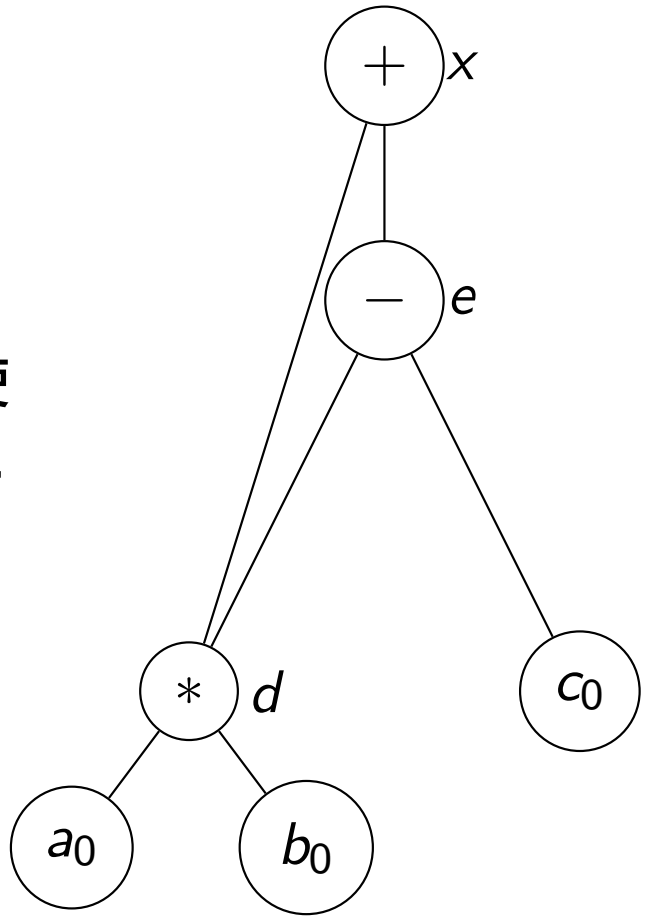



```
d=a*b
y=d+c
e=d-c
x=d+e
```

不要な変数を上からたどって消していく。
$e$,$d$ は直接使わないが、$x$ の計算に必要なので消去できない。

# 不要コードの消去

これ以降、$x$ しか使用しないとすると
$\mathrm{y} = \mathrm{d} + \mathrm{c}$ は消去

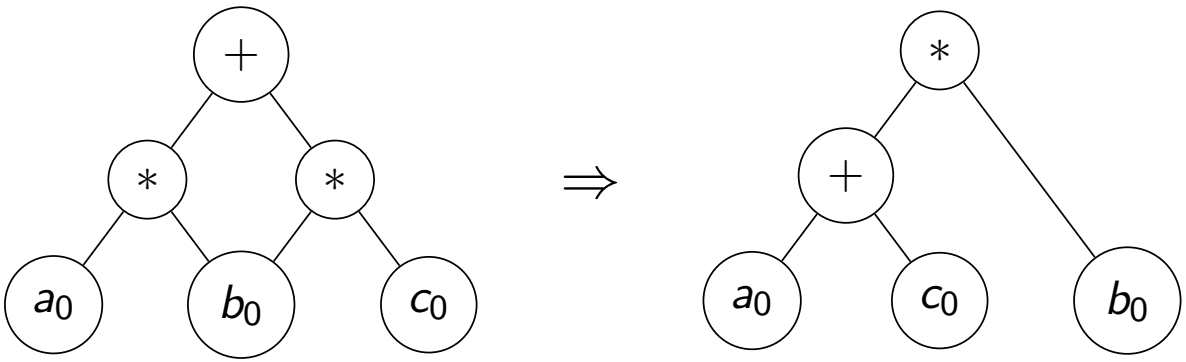



```
d=a*b

e=d-c
x=d+e
```

不要な変数を上からたどって消していく。
$e$,$d$ は直接使わないが、$x$ の計算に必要なので消去できない。

# 代数的性質




```
t1=a*b
t2=b*c
t3=t1+t2
```

```
t1=a+c
t2=t1*b
```

分配則=DAG 上でのパターンマッチング

# 大域的コード最適化

基本ブロックを越える最適化

- ループ最適化
- 手続き呼び出し最適化
- データフロー解析
- 静的単一代入 (SSA) 解析

# ループ最適化

ループ内のコード実行回数: ループ外のコードよりも一般に多い
ループ内のコードは慎重に最適化
本当に毎回実行しなければならないか？できるだけ効率よく実行

## ループ不変式の移動

ループ不変式: ループ内で評価が変わらない式

ループ内に `t` $:= a_1 \oplus a_2$ という式があり，この定義式を $d$ とする．

($\oplus$ は任意の２項演算子) $a_i$ に対して以下の少なくとも１つの条件がなりたつとき，ループ不変式である．

1. $a_i$ は定数
2. $a_i$ のすべての定義はループの外
3. $a_i$ のうちの１つだけが，$d$ に到達し，それはループ不変式の定義

例：`1 c=a/b` において，いずれの例においても `a`, `b` の定義は `do`$\cdots$`while` の外．２番めの場合は `n<=0` のときに注意

# 例

## 例

```
i=1;                                  i=1;t=a/b;
do{                                   do{
  c=a/b;x=i*c;...i++;                   c=t;x=i*c;...i++;
} while(i<n);                    ⇒    } while(i<n);


                                      if(n>1) {
                                        t=a/b;
 for(i=1;i<n;i++) {                     for(i=1;i<n;i++) {
   c=a/b;x=1*c; ...                       c=t;x=i*c;...
 }                                      }
                                  ⇒   }
```

ループ内の演算の強さの軽減
強い演算：時間のコストが大きい演算
足し算 < 掛け算 (加算器 < 乗算器)
例 6.9

# 例

## 例 6.8

```
i=1;
do{
  c=a/b;x=i*c;...i++;
} while(i<n);
```

⇒

```
i=1;t=a/b;
do{
  c=t;x=i*c;...i++;
} while(i<n);
```

```
for(i=1;i<n;i++) {
  c=a/b;x=1*c; ...
}
```

⇒

```
if(n>1) {
  t=a/b;
  for(i=1;i<n;i++) {
    c=t;x=i*c;...
  }
}
```

## 例 6.9

```
for(i=1;i<n;i++)
  a=a+b*i;
```

⇒

```
t=0;
for(i=1;i<n;i++) {
  t=t+b;a=a+t;
}
```

# ループ最適化

## 帰納変数消去

帰納変数 (ループを制御する変数) は減らす。

参照数の減少

```
t=b;
for(i=1;i<n;i++) {
  a=a+t;t=t+b;
}
```

$\Rightarrow$

```
m=n*b;
for(t=b;t<m;t=t+b)
  a=a+t;
```

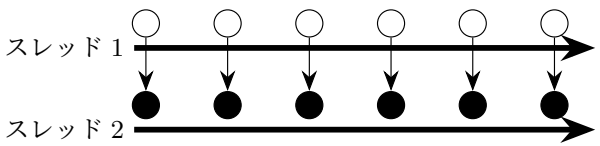
# ループ融合・ループ分離

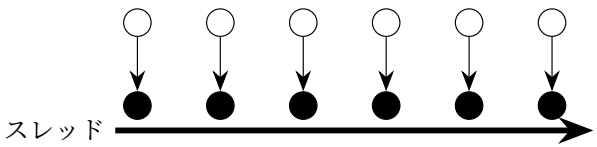
## ループ融合 (Loop fusion)

```
for(i=1;i<=N;i++)
  b[i]=c[i];
for(j=1;j<=N;j++)
  a[j]=b[j];
```

⇒

```
for(p=1;p<=N;p++) {
  a[p]=b[p]=c[p];
}
```




c[p] から読み出した値をキャッシュ(レジスタ) から失うことなくコピーできる.

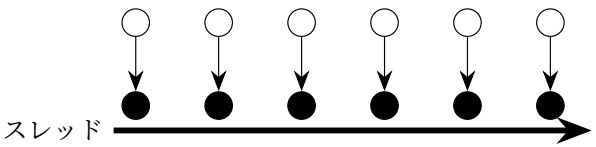
# ループ融合・ループ分離

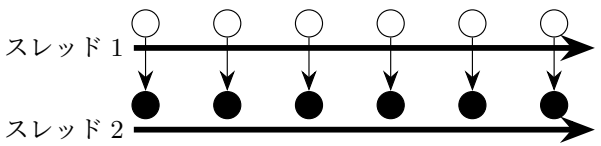
## ループ分離 (Loop fisson)

```
for(i=1;i<=N;i++)
  b[i]=c[i];
for(j=1;j<=N;j++)
  a[j]=b[j];
```

⇒

```
for(p=1;p<=N;p++) {
  a[p]=b[p]=c[p];
}
```




2 つのループは独立に計算することができる．a[i] から読み出した値を独立に利用できる．

# ループ展開とソフトウェアパイプライン

```
for(i=1;i<1000;i++) {
  a[i]=a[i-1]+1; // Ai
  b[i]=a[i]+b[i-1]; // Bi ⇒
  c[i]=b[i]+2;
}
```

```
a[1]=a[0]+1; // A1
b[1]=a[1]+b[0]; // B1
a[2]=a[1]+1; // A2
for(i=3;i<998,i++) {
  c[i-2]=b[i-2]+2; //Ci-2
  b[i-1]=a[i-1]+b[i-2]; //Bi-1
  a[i]=a[i-1]; // Ai
}
```

# 手続き呼出し最適化

インライン展開:
手続き呼出し=駆動レコードを用いてスタック上に情報を保存
操作としてはコストが大きい
スタック操作：実メモリへの正確な順序での書き込みを要求
短い手続きや関数であればそのまま置き換えてしまった方が効率がよい。

末尾再帰:
手続の最後の再帰よびだしは、goto に置き換え可能。
呼出しから復帰しても駆動レコード上の局所環境を使って実行する命令が存在しない。局所領域も再利用可能

# 末尾再帰の例

```
_Bool ans;
int a[10000];
void Bsearch(int x,int i,int j){
  int m;
  if (i>j) ans=false;
  else { m=(i+j)/2;
    if(x>a[m])  Bsearch(x,m+1,j);
    else if(x<a[m])  Bsearch(x,i,m-1);
    else ans=true;
  }
}
```

⇒

```
_Bool ans;
int a[10000];
void Bsearch(int x,int i,int j){
  int m;
 L:
  if (i>j) ans=false;
  else { m=(i+j)/2;
    if(x>a[m])  {i=m+1;goto L;}
    else if(x<a[m])  {j=m-1;goto L;}
    else ans=true;
  }
}
```

⇓

```
_Bool ans;
while(true){
  if(i>j) {ans=false;break;}
  else { m=(i+j)/2;
    if(x>a[m]) i=m+1;
    else if(x<a[m]) j=m-1;
    else {ans=true;break;}
  }
}
```

# データフロー解析

(講義資料 6.4: 詳細については省略)

基本ブロック $B$ に対するデータフロー

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

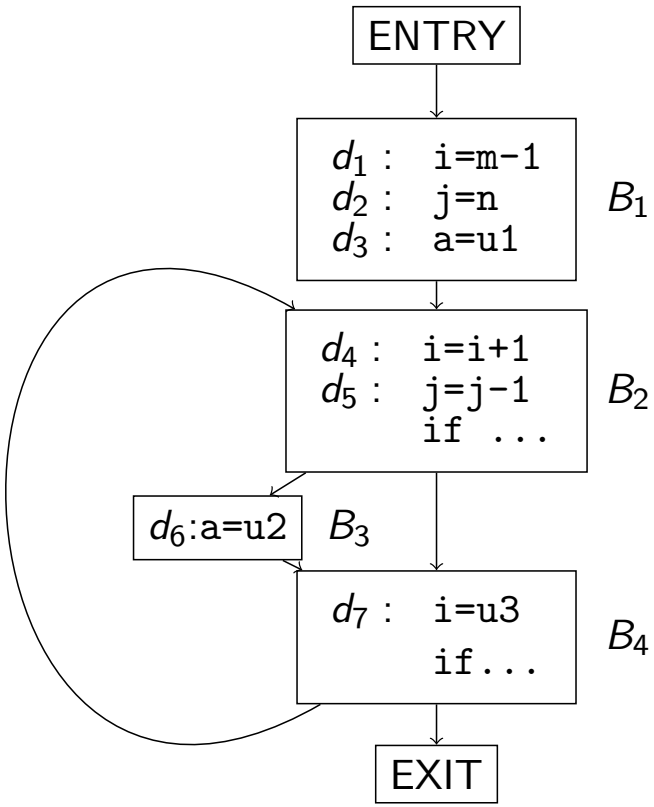
プログラム上での位置 (行番号)：

| | |
|---|---|
| $in[B]$ | $B$ に入力されるデータ |
| $out[B]$ | $B$ から出力されるデータ |
| $gen[B]$ | $B$ で定義されるデータ |
| $kill[B]$ | $B$ で上書きされるデータ |

入力から出力への制御がきまっていればよいので、基本ブロック単位で考えればよい。
ループ：データフロー方程式を解いて全体的に必要なデータを計算する。

# データフロー方程式

## 定義到達可能性解析：




$$gen_{B_1} = \{d_1, d_2, d_3\} \quad kill_{B_1} = \{d_4, d_5, d_6, d_7\}$$

$$gen_{B_2} = \{d_4, d_5\} \quad kill_{B_2} = \{d_1, d_2, d_7\}$$

$$gen_{B_3} = \{d_6\} \quad kill_{B_3} = \{d_3\}$$

$$gen_{B_4} = \{d_7\} \quad kill_{B_4} = \{d_1, d_4\}$$

$$IN[B_2] = OUT[B_1] \cup OUT[B_4]$$

$$OUT[B_2] = gen_{B_2} \cup (IN[B_2] - kill_{B_2})$$

(Aho ら：Compilers より引用)

# データフロー方程式

$$
\begin{aligned}
OUT[B_1] &= gen_{B_1} \\
IN[B_2] &= OUT[B_1] \cup OUT[B_4] \\
OUT[B_2] &= gen_{B_2} \cup (IN[B_2] - kill_{B_2}) \\
IN[B_3] &= OUT[B_2] \\
OUT[B_3] &= gen_{B_3} \cup (IN[B_3] - kill_{B_3}) \\
IN[B_4] &= OUT[B_3] \cup OUT[B_2] \\
OUT[B_4] &= gen_{B_4} \cup (IN[B_4] - kill_{B_4})
\end{aligned}
$$

# データフロー方程式の計算

**到達可能性解析：**

XXXX XXX:$d_1, d_2, \cdots, d_7$ が集合に入っているとき 1、入ってないとき 0

| Block $B$ | OUT$[B]^0$ | IN$[B]^1$ | OUT$[B]^1$ | IN$[B]^2$ | OUT$[B]^2$ |
|---|---|---|---|---|---|
| $B_1$ | 111 0000 | 000 0000 | 111 0000 | 000 0000 | 111 0000 |
| $B_2$ | 000 1100 | 111 0001 | 001 1100 | 111 0111 | 001 1110 |
| $B_3$ | 000 0010 | 000 1100 | 000 1110 | 001 1100 | 000 1110 |
| $B_4$ | 000 0001 | 000 1110 | 000 0111 | 001 1110 | 001 0111 |

変化しなくなるまでくりかえす。

$OUT[B_4]$=001 0111:

本ブロックでは $d_3, d_5, d_6, d_7$ が EXIT が到達する可能性がある.

# データフロー

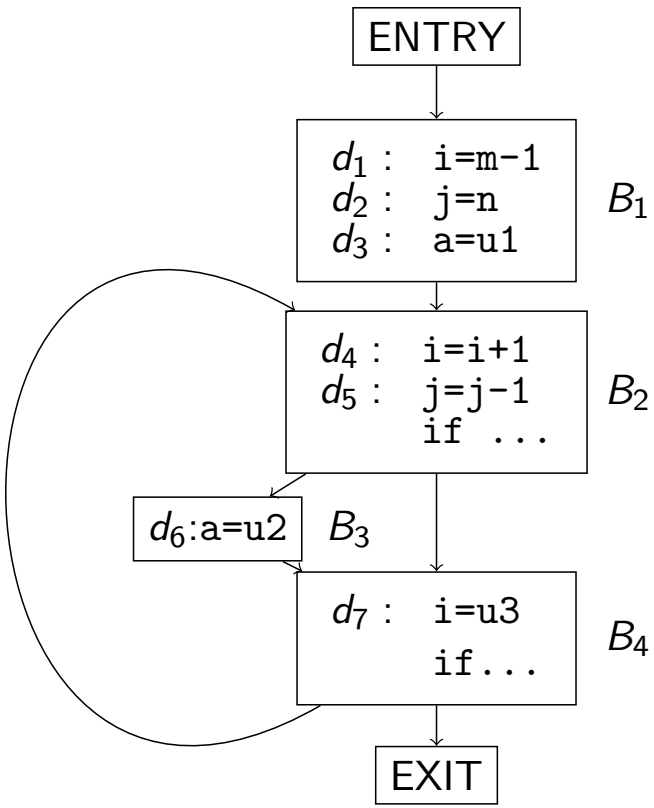
**生存解析**

$Use[B]$: $B$ で再定義される前に資料される変数の集合
$Def[B]$: $B$ で定義される変数の集合

$$
\begin{aligned}
In[B] &= Use[B] \cup (Out[B] - Def[B]) \\
Out[B] &= \cup_{S \in Succ[B]} In[S]
\end{aligned}
$$

$Succ[B]$: フローグラフにおいて $B$ から直接フローがあるブロック

# データフロー解析




$$Def_{B_1} = \{i, j, a\}$$
$$Use_{B_1} = \{m, n, u_1\}$$

$$Def_{B_2} = \{i, j\}$$
$$Use_{B_2} = \{i, j\}$$

$$Def_{B_3} = \{a\}$$
$$Use_{B_3} = \{u2\}$$

$$Def_{B_4} = \{i\}$$
$$Use_{B_4} = \{u3\}$$

# データフロー解析（生存解析）

$(a, i, j, m, n, u1, u2, u3)$ とすると

| | $OUT$ | $IN$ | $OUT$ | $IN$ | $OUT$ | $IN$ |
|---|---|---|---|---|---|---|
| $B_1$ | (00000000) | (00011100) | (01100000) | (00011100) | (01100010) | (00011100) |
| $B_2$ | (00000000) | (01100000) | (01100010) | (01100010) | (01100001) | (01100010) |
| $B_3$ | (00000000) | (00000010) | (00000001) | (00000010) | (00100001) | (10100001) |
| $B_4$ | (00000000) | (00000001) | (01100000) | (00100001) | (01100010) | (00100001) |

フローグラフの基本ブロックにおいてどの変数が影響をうけるかを解析

| | $IN$ | $OUT$ |
|---|---|---|
| $B_1$ | $\{m, n, u1\}$ | $\{i, j, u2\}$ |
| $B_2$ | $\{i, j, u2\}$ | $\{i, j, u3\}$ |
| $B_3$ | $\{a, j, u3\}$ | $\{j, u3\}$ |
| $B_4$ | $\{j, u3\}$ | $\{i, j, u2\}$ |